

2. Place & route with nextpnr-ecp5:

The nextpnr-ecp5 manpage documents LPF usage via `--lpf`.

Example:

```
nextpnr-ecp5 --25k --package CABGA381 \
  --json build/blinky.json \
  --lpf constraints/board.lpf \
  --textcfg build/blinky.config
```

3. Bitstream generation

Depending on your setup, the final bitstream pack step may be handled through trellis utilities invoked by nextpnr outputs. (In many practical guides, you'll see `.config/bitstream` conversions as part of the tooling chain.)

The important concept: Project Trellis documents ECP5 bitstream/architecture so open tools can generate valid configurations.

Uploading your bitstream: SRAM vs flash

Two common modes:

Load into SRAM (volatile)

- FPGA config disappears when power is removed
- Good for fast iteration

Write to flash (non-volatile)

- FPGA boots into your design on power-up
- Good once you trust your build

iCE40 programming with iceprog

A practical description: `iceprog` is a simple programming tool typically invoked after converting a bitstream to `.bin` with `icepack`.

Typical usage (exact command depends on your programmer/board):

```
iceprog build/blinky.bin
```

Universal programmer: openFPGALoader

openFPGALoader is an open-source utility for programming FPGAs across vendors, supporting devices including Lattice iCE40 and ECP5, and can load to SRAM or write to flash depending on options.

Typical pattern:

```
openFPGALoader -b <board_name> build/
blinky.bit
```

```
openFPGALoader -b <board_name> -f build/
blinky.bit
```

Board aliases and SRAM/flash examples are documented in its boards compatibility pages.

Fast Track reality check: the #1 upload failure is usually not “bad HDL.” It’s:

- wrong cable/driver permissions
- wrong board target option
- wrong file format (bin vs bit)
- wrong constraints (so your LED pin isn’t what you think it is)

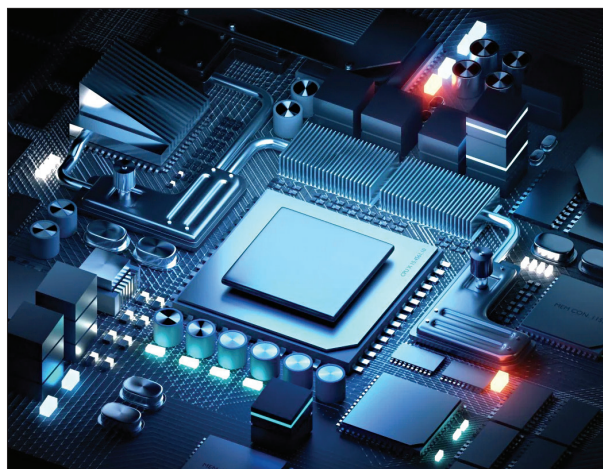
Debug workflow: LEDs, UART, and the “logic analyzer mindset”

Your first debugging tools are hilariously primitive – and extremely effective:

LEDs

Make your design “speak” using patterns:

- blink fast for “I’m alive”
- blink slow for “I passed init”
- pulse for “I saw button press”
- binary count on multiple LEDs if you have them



UART (coming soon in Chapter 11)

Once you can output serial text, debugging becomes 10× easier. Your UART TX from Chapter 7 becomes a real hardware logger.

Logic analyzer thinking

Even without a physical logic analyzer, you can debug by designing “visibility points” into your RTL:

- tap internal signals into output pins (temporarily)
- route internal state to LEDs
- expose a debug bus (even 4 pins help)

Timing report basics: what “met timing” means

You’ll see nextpnr output talk about timing. You don’t need to become a timing engineer today, but you should absorb this:

- Routing and logic take time.
- Your clock period is the deadline.
- If the slowest path in your design takes longer than the clock period, your design may fail.

This is why nextpnr being timing-driven matters (it’s designed as a timing-driven place and route tool).

Beginner win: with blinky, timing is usually fine. But once you do UART, video, or faster clocks, timing becomes your new boss. 